SOFTWARE ENGINEERING MINI-PROJECT

Software Requirements Specification

Task Manager Application — SRS

Document control	Value
Project	Task Manager Application v1.0
Version	1.0
Prepared / revised	15 October 2026
Status	Working baseline —approved
Technology baseline	Next.js (React), Node.js/Express REST API, MongoDB/Mongoose
Team	Akash Kiran Vernekar (PES1UG24AM338); Vismay H S (PES1UG24AM325); Basawaraj Panchal (PES1UG24AM350); Trisha G Shetty (PES1UG24AM308)
Submission deadline	30 October 2026

Control note: These documents define the proposed scope, architecture and verification plan. They do not claim the application is already implemented, deployed, tested, or approved. Update revision history, actual Jira keys, build status, test evidence and sign-offs as work is completed.

Document Control and Revision History

Version	Date	Owner	Change
1.0	15-Oct-2026	Project team	Consolidated baseline replacing earlier draft; aligned functional IDs, stack, acceptance criteria, data model, RTM and testing references.

Approval Status

Approval is pending. Each team member and the course evaluator should review the scope, constraints and acceptance criteria before this version is treated as approved.

Role	Member	Approval / date
Team member	Akash Kiran Vernekar (PES1UG24AM338)	Pending
Team member	Vismay H S (PES1UG24AM325)	Pending
Team member	Basawaraj Panchal (PES1UG24AM350)	Pending
Team member	Trisha G Shetty (PES1UG24AM308)	Pending
Course instructor	TBD	Pending

Contents

1. Introduction

2. Overall Description

3. External Interface Requirements

4. Functional Requirements

5. Non-Functional and Security Requirements

6. Acceptance Criteria

7. System Models

8. Requirements Traceability Matrix

9. Assumptions, Dependencies and Constraints

10. Data Requirements

11. Out of Scope and Future Scope

12. Jira, Git and Change Control

Appendix A. Glossary

1. Introduction

1.1 Purpose

This SRS defines the agreed candidate requirements for a browser-based task manager for small teams. It is the source of truth for design in SAD v1.0 and test planning in STP v1.0. Requirement IDs in this document are stable keys used in Jira issues, pull requests, tests and the requirements traceability matrix (RTM).

1.2 Scope

The v1.0 product supports user registration and authentication, user profile management, projects and project membership, task CRUD and assignment, a three-column Kanban board, comments, in-app assignment/due-date notifications, task search and filters, a project summary dashboard and CSV export. It is intended as a semester-scale demonstration of professional engineering workflow.

Hosting infrastructure administration, offline operation, third-party email provider internals and the future-scope items in Section 11 are excluded from v1.0. Next.js is the React-based web application framework; a separate Node.js/Express REST API is the backend boundary and MongoDB is the persistence layer.

1.3 Intended Audience

Project team, QA/test engineers, product owner, course instructor/evaluators, and maintainers.

1.4 Definitions

CRUD: create/read/update/delete; JWT: JSON Web Token; RBAC: role-based access control; RTM: Requirements Traceability Matrix; REST: Representational State Transfer; UAT: user acceptance testing; CI: continuous integration; CD: continuous delivery/deployment.

2. Overall Description

2.1 Product Perspective

The browser UI is implemented with Next.js and React. It calls an Express REST API using JSON over HTTPS. The API validates identity, authorization, business rules and request data before using Mongoose to read/write MongoDB. The browser must never connect directly to MongoDB. External integrations are not required for the baseline.

2.2 User Classes and Permissions

Role	Primary permissions	Restrictions
Admin	Manage user accounts and roles; full system administration.	All admin actions must be authenticated and audited; do not expose password hashes.
Project/Team Lead	Create projects, manage project members, create/edit/delete project tasks, assign tasks, view project reports.	Must be a member/owner of the project or Admin.
Team Member	View projects they belong to, view project tasks, update permitted task status, add comments, view their notifications.	Cannot manage other users or access projects/tasks outside their membership.

Role rules are enforced on the server, not only by hiding UI controls. Project membership is an authorization requirement in addition to the coarse account role. Exact admin onboarding and role-change workflow must be agreed before deployment; do not allow public registration to self-assign Admin.

2.3 Operating Environment

- Modern Chrome, Firefox and Edge on desktop/mobile form factors; internet connectivity required.
- Next.js/React web client; Node.js/Express API; MongoDB accessed through Mongoose.
- Local development and an isolated test database are required; deployment target is selected by the team and is not prescribed by this SRS.

2.4 Product Functions

- Register, login, logout/session expiry and profile update.
- Create and manage projects and project members.
- Create, update, delete, assign and change status of tasks.
- Present tasks grouped by To Do, In Progress and Done.
- Create task comments and in-app notifications.
- Search/filter tasks; show summary counts and overdue tasks; export project task data as CSV.

2.5 Constraints

- Project tracking, defects, scope decisions and sprint-level work are recorded in Jira.
- Source control uses GitHub with feature branches and pull-request review.
- Four-person team and submission deadline of 30 October 2026.

- Stack baseline: Next.js/React frontend, Node.js/Express REST API, MongoDB/Mongoose.
- Credentials and secrets are environment variables and must not be committed to the repository.

3. External Interface Requirements

3.1 User Interface

Responsive pages: registration/login, dashboard, project list/detail, Kanban board, task create/edit form, task comments, notification list and profile. Forms provide field-level validation and readable errors. Core functions must be usable using keyboard navigation and have adequate contrast and readable text.

3.2 API Interface

REST endpoints use JSON request/response bodies and standard HTTP methods/status codes. Protected endpoints require a valid non-expired bearer JWT in the Authorization header. API errors use a common shape {"errorCode":"...","message":"...","timestamp":"ISO-8601"}; error responses must not leak secrets or stack traces.

Area	Baseline endpoints (illustrative contract)
Auth	POST /api/auth/register; POST /api/auth/login; GET /api/auth/me; PATCH /api/users/me
Projects	GET/POST /api/projects; GET/PATCH/DELETE /api/projects/{projectId}; membership endpoints under /api/projects/{projectId}/members
Tasks	GET/POST /api/projects/{projectId}/tasks; GET/PATCH/DELETE /api/tasks/{taskId}; PATCH /api/tasks/{taskId}/status
Collaboration	GET/POST /api/tasks/{taskId}/comments; GET /api/notifications; PATCH /api/notifications/{notificationId}/read
Search/report	GET /api/tasks with query filters; GET /api/projects/{projectId}/summary; GET /api/projects/{projectId}/tasks/export.csv

Endpoint names are the proposed contract for implementation. Before coding, put the final OpenAPI or endpoint contract in the repository and keep API handlers, UI clients, SAD and tests aligned if paths change.

3.3 Database Interface

Only backend repository/data-access modules may use Mongoose to access MongoDB. Use schema validation, unique email index, appropriate indexes for projectId, assigneeIds and status, and createdAt/updatedAt timestamps. Use a separate database and synthetic test fixtures for automated tests.

3.4 Communications

- HTTPS/TLS 1.2 or later in deployed environments; local development may use localhost.
- JSON UTF-8 for normal API payloads; CSV response for export.
- No password, raw JWT, private environment values or sensitive request bodies in logs.

4. Functional Requirements

Each requirement is normative and testable. “Shall” indicates a required v1.0 behavior. Requirements TM-F-020..023 and TM-F-030..032 are individually defined below to ensure complete traceability.

ID	Feature	Priority	Requirement	Acceptance criterion	Test ref
TM-F-001	Registration	High	The system shall allow a new user to register with name, valid email and password.	Valid registration creates one account; duplicate email returns 409; invalid fields return 400. Password is bcrypt-hashed and never persisted in plaintext.	TC-AUTH-01
TM-F-002	Login and session	High	The system shall authenticate registered users and issue a time-limited JWT.	Correct credentials return a token and expiry; wrong credentials return 401; expired/invalid token on protected route returns 401.	TC-AUTH-02, TC-SEC-02
TM-F-003	Profile management	Medium	The system shall allow an authenticated user to update permitted profile fields and change password.	Updated name is persisted; password change requires current authentication and new password hash works on next login.	TC-AUTH-03
TM-F-010	Create task	High	The system shall allow an authorized project member with Lead/Admin permissions to create a task with title, description, due date, priority, status, project and assignee(s).	Valid task appears in project task list/board and is persisted; missing title/project or invalid values return 400.	TC-TASK-01
TM-F-011	Edit/delete task	High	The system shall allow an authorized	Updated values persist; deletion removes	TC-TASK-02

ID	Feature	Priority	Requirement	Acceptance criterion	Test ref
			Lead/Admin to update task details and delete a task within an authorized project.	task from active lists; unauthorized access returns 403 and unknown ID returns 404.	
TM-F-012	Update task status	High	The system shall support To Do, In Progress and Done statuses and persist status changes.	Allowed transition is saved and shown in correct board column; unknown status returns 400.	TC-TASK-03
TM-F-020	Manage projects	High	The system shall allow an authenticated Lead/Admin to create and edit a project.	Project has unique identifier, name, description and owner; members can see only authorized projects.	TC-PROJ-01
TM-F-021	Manage project members	High	The system shall allow a project Lead/Admin to add or remove registered project members.	Membership update persists; invalid users/projects return 404/400; non-authorized mutation returns 403.	TC-PROJ-02, TC-SEC-03
TM-F-022	Assign tasks	High	The system shall allow an authorized Lead/Admin to assign one or more registered members of the project to a task.	Assignee IDs are valid project members; assignment appears on task detail; invalid/non-member assignee is rejected.	TC-BOARD-01
TM-F-023	Kanban board	High	The system shall display project tasks in To Do, In Progress and Done columns and reflect saved status changes.	A task appears in exactly one status column and remains there after refresh.	TC-BOARD-02, TC-E2E-01
TM-F-030	Assignment notification	Medium	The system shall create an in-app notification when a task is assigned to a user.	Notification identifies task/event and recipient; unrelated users cannot read it.	TC-NOTIF-01
TM-F-031	Due-date reminder	Medium	The system shall create an in-app due-date reminder according to a documented reminder rule.	Reminder rule (initial proposal: due within 24 hours and not Done) is configurable/testable; duplicate reminders are not repeatedly generated for the same task window.	TC-NOTIF-02
TM-F-032	Task comments	Medium	The system shall allow authorized project members to post and read task comments.	Comment is stored with task, author and timestamp; empty or oversized comment is rejected; unauthorized project access returns 403.	TC-COMM-01
TM-F-040	Search and filters	Medium	The system shall support keyword search and filters for status, priority and assignee on authorized tasks.	Combined filters return only matching visible tasks; empty result has clear empty state.	TC-SEARCH-01
TM-F-041	Dashboard summary	Medium	The system shall display project task counts by status and count of overdue unfinished tasks.	Summary equals a direct calculation from authorized project data and excludes inaccessible projects.	TC-REPORT-01
TM-F-042	CSV export	Medium	The system shall export an authorized project task list in CSV format.	Header and row values match selected project/filter scope; CSV cells are escaped safely; unauthorized project export returns 403.	TC-REPORT-02

5. Non-Functional and Security Requirements

ID	Attribute	Priority	Requirement	Verification
TM-NF-001	Performance	High	Under a documented normal-load test profile, at least 95% of measured API requests shall complete within 2 seconds (p95 ≤ 2 s), excluding external provider wait where applicable.	k6/JMeter run against staging; report sample size, environment and percentile.
TM-NF-002	Reliability	Medium	The intended operational availability target is at least 99% per month excluding planned maintenance.	Operational target; cannot be proven by a short course-project test. Report observed monitoring window and limitations.
TM-NF-003	Credential protection	High	Passwords shall be stored using salted bcrypt hashes; plaintext passwords and JWTs shall not be logged or returned except token returned to authenticated login client.	Database inspection, API response inspection and log review.
TM-NF-004	Usability/accessibility	Medium	Core flows shall be responsive on desktop/mobile and support keyboard navigation, readable text and adequate color contrast.	Manual checklist on target browsers/viewports; automated checks support but do not replace manual review.
TM-NF-005	Concurrent load	Medium	The backend shall support 100 concurrent simulated users without noticeable degradation; use p95 latency and error rate to quantify the result.	Load test. Define request mix and success threshold in the test run report; do not claim pass without evidence.
PRJ-SR-001	Password storage	High	Passwords shall be bcrypt-hashed and never stored in plaintext or logged.	Storage and logs inspected; response omits passwordHash.
PRJ-SR-002	JWT security	High	Protected routes shall reject missing, malformed, expired and invalid JWTs with 401.	Automated API security tests.
PRJ-SR-003	Authorization/RBAC	High	Server-side role and project-membership checks shall prevent unauthorized access or mutation of projects, tasks, comments and notifications.	Non-member read/write returns 403 (or 404 where chosen to avoid resource enumeration); behavior is documented and consistent.
PRJ-SR-004	Transport security	High	All deployed client/API traffic shall use HTTPS/TLS 1.2 or later.	Verify deployment configuration/certificate and

ID	Attribute	Priority	Requirement	Verification
				redirect policy.
PRJ-SR-005	Input validation	High	The backend shall validate and normalize input and prevent NoSQL injection, XSS and unsafe CSV formula injection.	Negative/fuzz tests; render user text as text, never unsafely inject HTML.
PRJ-SR-006	Operational safeguards	High	Authentication endpoints shall be rate limited; secrets shall be injected via environment configuration; API errors shall not disclose stack traces.	Verify configured rate-limit behavior, repository secret scan and production error responses.
TM-NF-006	Maintainability/testability	Medium	API routes, services, data-access modules and UI components shall have clear responsibilities and repeatable automated checks.	PR review and CI results; changes include appropriate tests.

6. Acceptance Criteria

Release candidate is acceptable only when all High-priority functional requirements are implemented and verified; no critical/high-severity unresolved security or data-loss defects remain; regression tests pass; the RTM has evidence for all in-scope requirements; and the team has demonstrated the two principal end-to-end flows.

- Registration/login: new account, duplicate-email rejection, successful login, invalid/expired token rejection.
- Project/task flow: create project, add member, create/assign task, change status, refresh, confirm persistence and notification.
- Data correctness: validation, required references, owner/member boundaries, timestamp fields and indexes behave as designed.
- Search/report/export: filters compose correctly; summary counts match source records; CSV encoding and escaping are checked.
- Performance and uptime: report measured evidence and environment; where shared staging makes targets infeasible, log deviation and instructor-approved disposition instead of marking pass without evidence.

7. System Models

UML use-case diagrams are maintained with the design assets. The SRS logical actors/use cases are:

Actor	Use cases
Guest	Register, log in
Admin	Manage users/roles; all permitted project/task administration
Project/Team Lead	Create project, manage members, create/edit/delete/assign tasks, view report, export CSV
Team Member	View authorized board/tasks, update permitted status, comment, view/read own notifications
External/CI tooling	Run automated unit, integration, end-to-end and load tests; not a product end-user actor

The SAD provides the authoritative architecture, component model, data relationships and sequence flows. Proposed diagrams: use-case overview; project/task entity relationship model; component architecture; registration/login sequence; create-and-assign-task sequence; task status-update and notification sequence.

8. Requirements Traceability Matrix (RTM)

Requirement ID	Short name	SRS section	Design mapping	Verification ID	Status
TM-F-001	Registration	SRS §4	SAD modules: UI + corresponding service	TC-AUTH-01	Planned
TM-F-002	Login and session	SRS §4	SAD modules: UI + corresponding service	TC-AUTH-02, TC-SEC-02	Planned
TM-F-003	Profile management	SRS §4	SAD modules: UI + corresponding service	TC-AUTH-03	Planned
TM-F-010	Create task	SRS §4	SAD modules: UI + corresponding service	TC-TASK-01	Planned
TM-F-011	Edit/delete task	SRS §4	SAD modules: UI + corresponding service	TC-TASK-02	Planned
TM-F-012	Update task status	SRS §4	SAD modules: UI + corresponding service	TC-TASK-03	Planned
TM-F-020	Manage projects	SRS §4	SAD modules: UI + corresponding service	TC-PROJ-01	Planned
TM-F-021	Manage project members	SRS §4	SAD modules: UI + corresponding service	TC-PROJ-02, TC-SEC-03	Planned
TM-F-022	Assign tasks	SRS §4	SAD modules: UI + corresponding service	TC-BOARD-01	Planned
TM-F-023	Kanban board	SRS §4	SAD modules: UI + corresponding service	TC-BOARD-02, TC-E2E-01	Planned
TM-F-030	Assignment notification	SRS §4	SAD modules: UI + corresponding service	TC-NOTIF-01	Planned
TM-F-031	Due-date reminder	SRS §4	SAD modules: UI + corresponding service	TC-NOTIF-02	Planned
TM-F-032	Task comments	SRS §4	SAD modules: UI + corresponding service	TC-COMM-01	Planned

Requirement ID	Short name	SRS section	Design mapping	Verification ID	Status
TM-F-040	Search and filters	SRS §4	SAD modules: UI + corresponding service	TC-SEARCH-01	Planned
TM-F-041	Dashboard summary	SRS §4	SAD modules: UI + corresponding service	TC-REPORT-01	Planned
TM-F-042	CSV export	SRS §4	SAD modules: UI + corresponding service	TC-REPORT-02	Planned
TM-NF-001	Performance	SRS §5	SAD §3/§4 cross-cutting	TC-PERF-01	Planned
TM-NF-002	Reliability	SRS §5	SAD §3/§4 cross-cutting	OPS-MON-01	Planned
TM-NF-003	Credential protection	SRS §5	SAD §3/§4 cross-cutting	TC-SEC-01	Planned
TM-NF-004	Usability/accessibility	SRS §5	SAD §3/§4 cross-cutting	TC-UX-01	Planned
TM-NF-005	Concurrent load	SRS §5	SAD §3/§4 cross-cutting	TC-PERF-02	Planned
PRJ-SR-001	Password storage	SRS §5	SAD §3/§4 cross-cutting	TC-SEC-01	Planned
PRJ-SR-002	JWT security	SRS §5	SAD §3/§4 cross-cutting	TC-SEC-02	Planned
PRJ-SR-003	Authorization/RBAC	SRS §5	SAD §3/§4 cross-cutting	TC-SEC-03	Planned
PRJ-SR-004	Transport security	SRS §5	SAD §3/§4 cross-cutting	TC-SEC-04	Planned
PRJ-SR-005	Input validation	SRS §5	SAD §3/§4 cross-cutting	TC-SEC-05	Planned
PRJ-SR-006	Operational safeguards	SRS §5	SAD §3/§4 cross-cutting	TC-SEC-06	Planned
TM-NF-006	Maintainability/testability	SRS §5	SAD §3/§4 cross-cutting	TC-CI-01	Planned

Status values at baseline: Planned. Change to Pass/Fail/Blocked only with dated execution evidence or explicit review evidence. Keep exact IDs and document names aligned in Jira, the SAD and STP.

9. Assumptions, Dependencies and Constraints

- Node.js, package manager, GitHub and a MongoDB development/test environment are accessible to all four contributors.
- A test/staging API and isolated seeded dataset are available before end-to-end/load tests.
- Users have a modern browser and network connection; offline mode is not required.
- The team confirms the Next.js frontend and separate Express API split before scaffolding.
- Reminder schedule, password policy, token expiry and initial project membership/onboarding rule must be recorded in Jira decisions before implementation.
- Scope changes require Jira issue update, SRS revision, impact review in SAD/STP and pull-request review.

10. Data Requirements

Entity / collection	Required fields (baseline)	Relationships / rules
User	_id, name, emailNormalized, passwordHash, role, createdAt, updatedAt	Email unique; role enum Admin/Lead/Member; passwordHash never returned in API.
Project	_id, name, description, ownerId, memberIds[], createdAt, updatedAt	ownerId references User; membership validated server-side.
Task	_id, title, description, status, priority, dueDate, projectId, assigneeIds[], createdAt, updatedAt	projectId references Project; each assignee must be a member; status/priority enums; useful indexes on projectId/status/assigneeIds/dueDate.
Comment	_id, taskId, authorId, text, createdAt	References Task and User; read/write requires project membership.
Notification	_id, userId, taskId, type, message, read, createdAt	Owns one recipient; type supports assignment/due-date; only recipient/admin authorized to access as defined.

MongoDB/Mongoose document references are logical foreign-key relationships; application services must maintain referential correctness. Define delete behavior (for example, cascade/archive task comments and notifications) before implementation and test it. Use timestamps and explicit validation; add indexes based on measured query patterns.

11. Out of Scope and Future Scope

- Task file attachments.
- Recurring tasks and calendar view.
- Email notifications in addition to in-app notifications.
- Dark mode.
- Detailed task activity/audit history beyond baseline timestamps.
- Offline-first usage, mobile native app, multi-tenant enterprise provisioning and external integrations.

12. Jira, Git and Change Control

- Create Jira Epics for Identity, Projects & Tasks, Kanban, Collaboration, Search/Reports, Quality/CI, and Documentation/Release. Link each story/bug to requirement IDs and test IDs.
- Use issue types Epic, Story/Task, Bug, Spike; every story has user value, acceptance criteria, estimate, assignee, priority and requirement ID.
- Branch convention: feature/TM-123-short-name, fix/TM-123-short-name, docs/TM-123-short-name. No routine direct commits to main.
- Pull requests target main; require at least one teammate review, linked Jira issue, lint/build/unit tests green, and a summary of test evidence.
- CI baseline: install dependencies from lockfile; lint; type/static checks if configured; unit and API integration tests; build Next.js/API; E2E smoke tests when stable. Keep DB secrets out of CI logs.
- Definition of Done: acceptance criteria met, tests added/updated and passing, lint/build clean, review resolved, docs and RTM updated, no known critical security/data defect, Jira issue updated.

Appendix A. Glossary

Next.js: React framework; Express: Node.js web/API framework; Mongoose: MongoDB object modeling/validation library; Kanban: visual workflow board; P95: 95th-percentile latency; synthetic data: generated non-production test records; CI/CD: continuous integration and continuous delivery/deployment.